HAMi-Core x DRA

—— 原生 DRA Driver 实践

杨守仁，第四范式研发工程师 & HAMi Approver

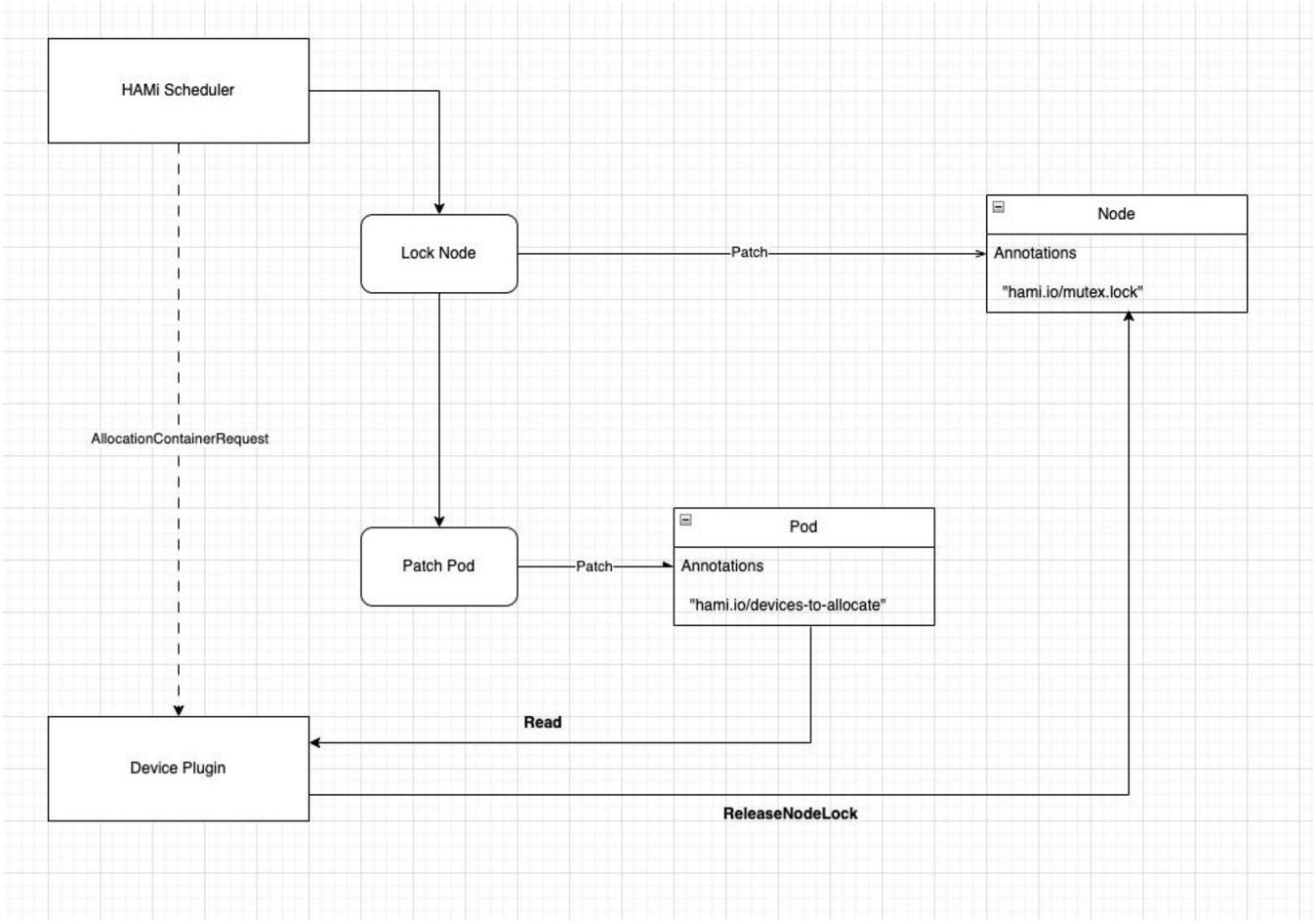
范式 PHANCY

背景和动机

- Device Plugin 的限制
  - 容器和设备具体信息缺失
- HAMi-Core 的隐藏特性
  - 带后缀的环境变量

背景和动机 — Device Plugin

- HAMi 的方案
  - 节点锁
    - Node Annotations
  - 设备信息
    - Pod Annotations
  - 问题: 5 mins nodelock
    - Issues: 810,1130,1368, …
    - PRs: 1117, 1244



背景和动机 — HAMi-Core

- HAMi-Core 的隐藏特性
  - CUDA_DEVICE_MEMORY_LIMIT
  - CUDA_DEVICE_MEMORY_LIMIT_{idx}
- Device Plugin 兼容?
  - Pod Annotations
  - 带后缀的 ResourceName
  - Quota 适配的问题

HAMi-core / src / multiprocess / multiprocess_memory_limit.c

Code Blame Executable File · 945 lines (862 loc) · 30.2 KB

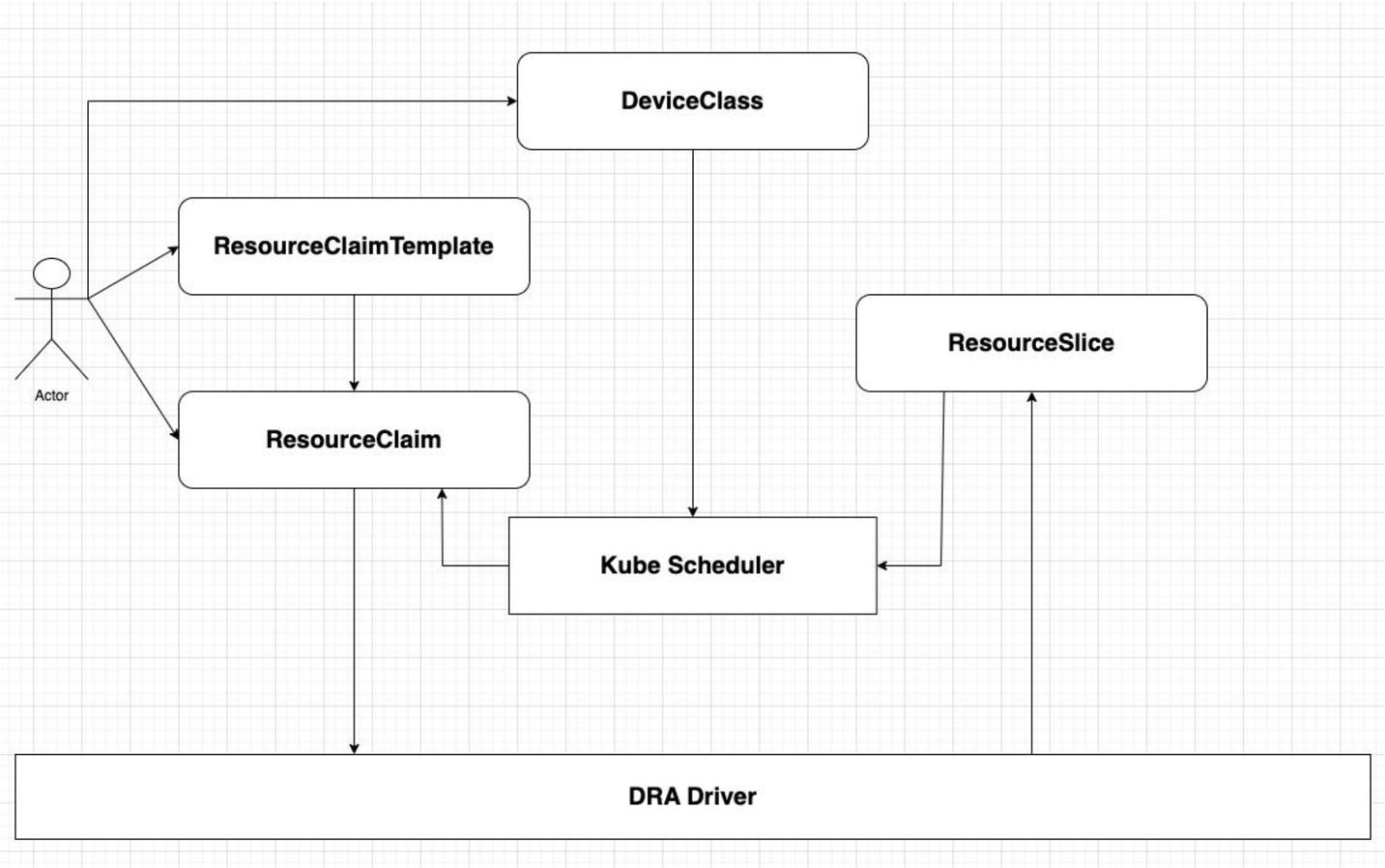
```c
void do_init_device_memory_limits(uint64_t* arr, int len) {
    size_t fallback_limit = get_limit_from_env(CUDA_DEVICE_MEMORY_LIMIT);
    int i;
    for (i = 0; i < len; ++i) {
        char env_name[CUDA_DEVICE_MEMORY_LIMIT_KEY_LENGTH] = CUDA_DEVICE_MEMORY_LIMIT;
        char index_name[8];
        snprintf(index_name, 8, "_%d", i);
        strcat(env_name, index_name);
        size_t cur_limit = get_limit_from_env(env_name);
        if (cur_limit > 0) {
            arr[i] = cur_limit;
        } else if (fallback_limit > 0) {
            arr[i] = fallback_limit;
        } else {
            arr[i] = 0;
        }
    }
}
```

HAMi-Core x DRA

- DRA 简介
- HAMi-Core DRA Driver 开发实践
- HAMi-Core DRA Driver 未来规划

HAMi-Core x DRA – DRA 简介

- DeviceClass
- **ResourceSlice**
- ResourceClaimTemplate
- **ResourceClaim**



HAMi-Core x DRA — 开发实践

- KEP-4815 DRA: Partitionable Devices
  - Kubernetes v1.33.0 – 2025.04
  - / 4815-dra-partitionable-devices /

Non-Goals

- Allow a user to allocate arbitrarily-sized partitions of a device. In other words, only those partitions explicitly defined by a `ResourceSlice` will be allocatable by a user. Arbitrary slicing is out of scope for this KEP.

HAMi-Core x DRA — 开发实践

- KEP-5075 DRA: Consumable Capacity
  - Kubernetes v1.34.0 – 2025.08
  -

```yaml
apiVersion: resource.k8s.io/v1
kind: ResourceClaim
metadata:
  name: double-gpu-0
spec:
  devices:
    - name: double-gpu-0-0
      exactly:
        deviceClassName: hami-core-gpu.project-hami.io
        capacity:
          requests:
            memory: "8Gi"
```

```yaml
apiVersion: resource.k8s.io/v1
kind: ResourceSlice
metadata:
  name: k8s-dra-driver-cluster-worker-hami-core-gpu.project-hami.ikffwb
spec:
  driver: hami-core-gpu.project-hami.io
  nodeName: k8s-dra-driver-cluster-worker
  pool:
    generation: 1
    name: k8s-dra-driver-cluster-worker
    resourceSliceCount: 1
  devices:
  - allowMultipleAllocations: true
    attributes:
      resource.kubernetes.io/pcieRoot:
        string: pci0000:80
      type:
        string: hami-gpu
    capacity:
      memory:
        requestPolicy:
          default: 11Gi
          validRange:
            max: 11Gi
            min: 1Mi
            step: 1Mi
        value: 11Gi
    name: hami-gpu-3
```

HAMi-Core x DRA — 开发实践

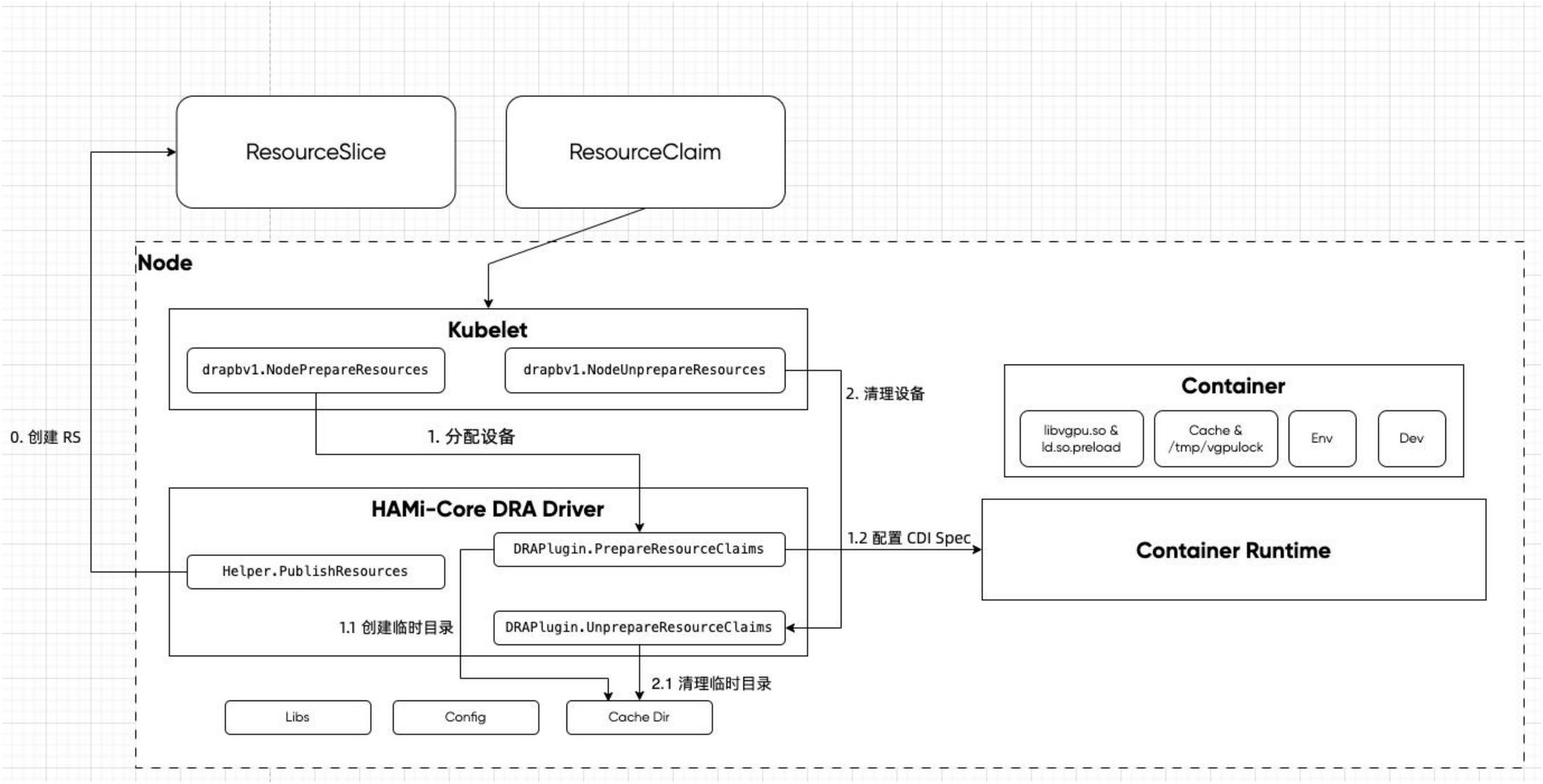
- k8s.io/dynamic-resource-allocation/kubeletplugin
- DRA Helper
  - **PublishResources**
- DRA Plugin
  - **PrepareResourceClaims**
    - 配置设备
    - CDI Spec
  - **UnprepareResourceClaims**
    - 清理临时文件

HAMi-Core x DRA — 开发实践

- HAMi-Core 配置
  - libvgpu.so 和 ld.so.preload
  - Cache 和 Lock 目录
  - 环境变量
    - CUDA_DEVICE_MEMORY_LIMIT_{idx}

- CDI API
  - ContainerEdits.Mounts
  - ContainerEdits.Env

HAMi-Core x DRA — 开发实践



HAMi-Core x DRA — 未来规划

- Repo 地址
  - https://github.com/Project-HAMi/k8s-dra-driver
- 设备配置
  - 对齐 HAMi NVIDIA Device Plugin 的 ConfigMap 提供的功能
  - 支持配置 HAMi-Core 相关的其它环境变量
- 监控功能
- 节点内的拓扑感知
  - NUMA
  - NVLink

HAMi-Core x DRA — 未来规划

- HAMi-DRA 联动
  - **异构设备的标准属性**
    - resource.kubernetes.io/pcieRoot
    - resource.kubernetes.io/pciBusID
    - **resource.project-hami.io/xxx ?**

dra-driver-cpu / pkg / driver / dra_hooks.go

Code Blame 478 lines (412 loc) · 17 KB

```go
140  func (cp *CPUDriver) createCPUDeviceSlices() [][]resourceapi.Device {
197              cpuDevice := resourceapi.Device{
198                  Name: deviceName,
199                  Attributes: map[resourceapi.QualifiedName]resourceapi.DeviceAttribute{
200                      "dra.cpu/numaNodeID": {IntValue: &numaNode},
201                      "dra.cpu/cacheL3ID":  {IntValue: &cacheL3ID},
202                      "dra.cpu/coreType":   {StringValue: &coreType},
203                      "dra.cpu/socketID":   {IntValue: &socketID},
204                      "dra.cpu/coreID":     {IntValue: &coreID},
205                      "dra.cpu/cpuID":      {IntValue: &cpuID},
206                      // TODO(pravk03): Remove. Hack to align with NIC (DRANet). We need some standard attribute to align other resources with CPU.
207                      "dra.net/numaNode": {IntValue: &numaNode},
208                  },
209                  Capacity: make(map[resourceapi.QualifiedName]resourceapi.DeviceCapacity),
210              }
211              devicesByNuma[numaNode] = append(devicesByNuma[numaNode], cpuDevice)
212          }
213      }
```

HAMI 设备插件新功能介绍

James 第四范式平台研发工程师

范式 PHANCY

目录

1.HAMi 昇腾插件集成 Volcano

2.资源名自动补齐

1.HAMi 昇腾插件集成 Volcano

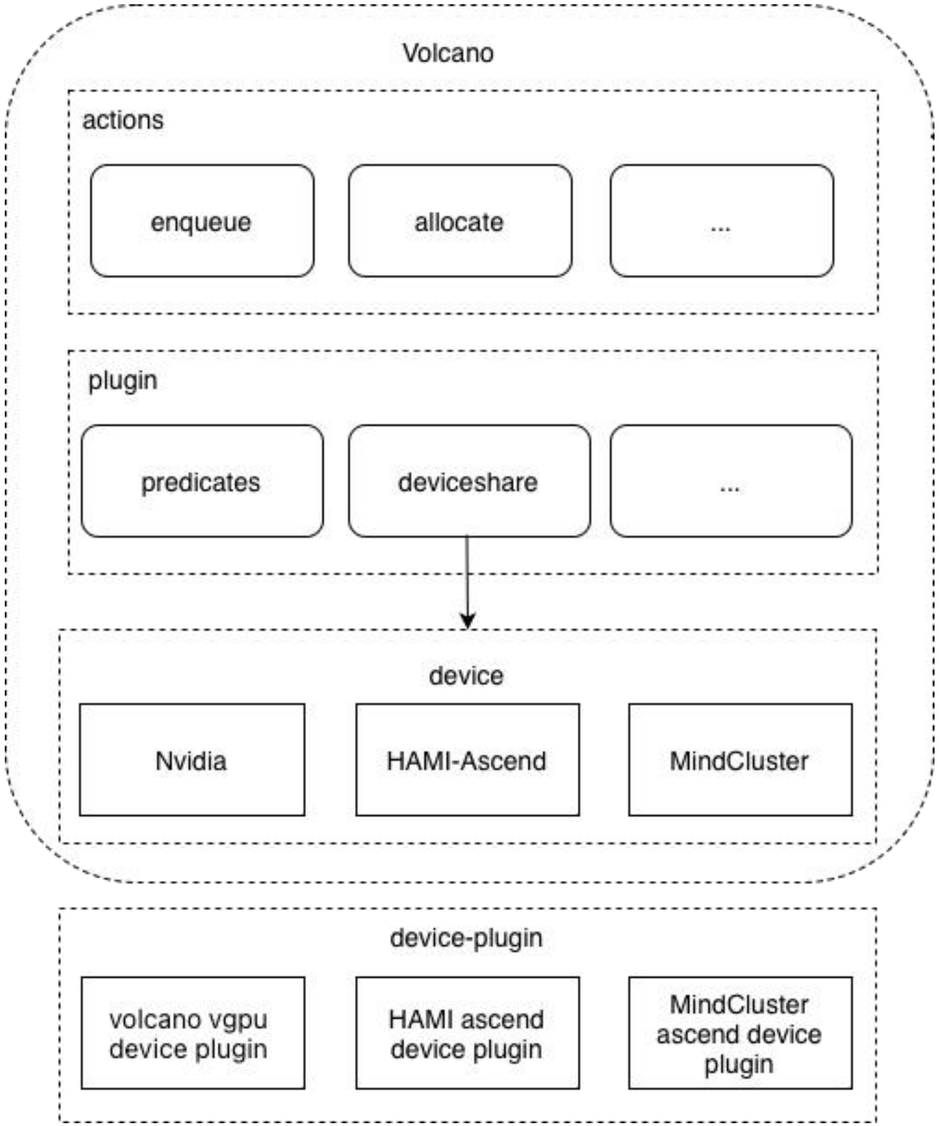
背景和现状

在 Kubernetes 中直接使用华为昇腾（Ascend）设备有如下问题：

- 调度模型默认以整卡独占为前提
- 资源抽象粒度偏粗
- 昇腾设备难以被纳入统一、细粒度的调度体系

Volcano × HAMi 架构

- 新增 HAMI Ascend device
  - 设备初始化
  - 设备筛选
  - 设备分配
- 在 share device pool 中注册 device



使用方式

修改 Volcano 部署配置

```yaml
kind: ConfigMap
apiVersion: v1
metadata:
  name: volcano-scheduler-configmap
  namespace: volcano-system
data:
  volcano-scheduler.conf: |
    actions: "enqueue, allocate, backfill"
    tiers:
    - plugins:
      - name: predicates
      - name: deviceshare
        arguments:
          deviceshare.AscendHAMiVNPUEnable: true   # enable ascend vnpu
          deviceshare.SchedulePolicy: binpack  # scheduling policy. binpack / spread
          deviceshare.KnownGeometriesCMNamespace: kube-system
          deviceshare.KnownGeometriesCMName: hami-scheduler-device
```

详细使用参考: https://github.com/volcano-sh/volcano/blob/master/docs/user-guide/how_to_use_vnpu.md

使用方式

在 Pod 的声明中使用昇腾资源

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: ascend-pod
spec:
  schedulerName: volcano
  containers:
    - name: ubuntu-container
      image: swr.cn-south-1.myhuaweicloud.com/ascendhub/ascend-pytorch:24.0.RC1-A2-1.11.0-ubuntu20.04
      command: ["sleep"]
      args: ["100000"]
      resources:
        limits:
          huawei.com/Ascend310P: "1"
          huawei.com/Ascend310P-memory: "4096"
```

详细使用参考: https://github.com/volcano-sh/volcano/blob/master/docs/user-guide/how_to_use_vnpu.md

2. 资源名自动补齐

背景和现状

device-plugin 只注册了 gpu 个数，没有注册显存等信息

用户监控不到显存等资源信息

mock-device-plugin

- 读取节点 annotation 解析设备信息
- 向 Kubernetes 注册设备资源

支持的设备及资源：

Devices	Mocking Resources
Nvidia GPU	nvidia.com/gpumem nvidia.com/gpumem-percentage nvidia.com/gpucores
Hygon DCU	hygon.com/dcumem
Ascend	huawei.com/Ascend{chip-name}-memory

https://github.com/Project-HAMi/mock-device-plugin

mock-device-plugin

```
allocatable:
  cpu: 39600m
  ephemeral-storage: 1071962728Ki
  hugepages-1Gi: "0"
  hugepages-2Mi: "0"
  hygon.com/dcumem: "39312"
  hygon.com/dcunum: "24"
  memory: "192139269379"
  pods: "110"
capacity:
  cpu: "40"
  ephemeral-storage: 1071962728Ki
  hugepages-1Gi: "0"
  hugepages-2Mi: "0"
  hygon.com/dcumem: "39312"
  hygon.com/dcunum: "24"
  memory: 198050532Ki
  pods: "110"
```

Q&A

AI for everyone

www.phancy.ai

范式
PHANCY